

Reflective Engine: A Local Inference-Time Framework for Iterative Self-Evaluation in Language Model Applications

Christian Morogan

Research-preview artifact paper

Evidence review date: 2026-05-31

Repository reviewed: TRXAlpha/Reflective-Engine, commit 9eba1ad

Abstract

Large language model applications often expose a single-pass interaction pattern: a prompt is sent to a model, a response is generated, and the response is returned to the user. This paper presents Reflective Engine, a local-first research prototype that wraps an Ollama-compatible language model in an iterative self-evaluation loop. The implementation adds model-selected reflection depth, self-scoring prompts, answer revision, convergence tokens, repeated-answer detection, and JSONL memory summaries without changing model weights.

The contribution of this paper is not a benchmark claim. The reviewed repository does not include a reproducible evaluation harness, benchmark datasets, or raw result logs. Instead, this paper makes artifact-grounded claims: the architecture is implemented, it can orchestrate iterative critique and revision around a local model, and it provides a concrete basis for future controlled evaluation. The paper also identifies limits of inference-time reflection, including dependence on the base model's knowledge, risk of self-reinforcing errors, weak self-grading reliability, and incomplete reproducibility support.

1. Introduction

Language models are frequently deployed as stateless text generators. In many applications, this is sufficient for short factual or stylistic tasks. For tasks that require careful reasoning, debugging, summarization, or multi-step explanation, single-pass generation has a practical weakness: the first answer is often returned before any explicit verification or revision step occurs.

Reflective Engine explores a simple engineering response to this problem. Instead of treating model improvement as training, fine-tuning, or reinforcement learning, it treats improvement as an inference-time orchestration problem. The application asks a model to produce an answer, rate or review that answer, revise it, and stop when a convergence condition is met or a user-specified loop budget is exhausted.

This design is intentionally modest. It does not claim that a smaller model becomes equivalent to a larger model. It does not claim guaranteed factual correction. It does not claim persistent learning in the sense of parameter updates. It provides a software architecture for iterative self-evaluation and a local implementation that can be inspected, modified, and evaluated.

2. Contributions

This paper makes four concrete contributions:

- It documents a local reflective-loop architecture implemented around an Ollama-compatible text generation endpoint.
- It describes a prototype that combines model-selected loop counts, model self-scoring, review prompts, improvement prompts, convergence tokens, repeated-answer detection, and JSONL memory.
- It distinguishes verified artifact claims from unverified performance claims.

- It includes a minimal baseline-versus-reflection evaluation scaffold and outlines what is still needed before making quantitative claims about accuracy, latency, energy use, or model-size equivalence.

3. Background And Related Work

Reflective Engine belongs to a broader family of test-time reasoning and self-improvement methods. Chain-of-Thought prompting showed that eliciting intermediate reasoning can improve performance on some reasoning tasks. Self-Consistency samples multiple reasoning paths and aggregates answers, improving several arithmetic and commonsense benchmarks at increased inference cost. Reflexion uses verbal feedback and memory to improve agent behavior across trials. Tree of Thoughts generalizes reasoning into search over intermediate "thought" states. Self-Refine uses iterative feedback and refinement without updating model weights.

Reflective Engine is closest in spirit to Self-Refine and Reflexion, but it is presented as an application-level local prototype rather than a new model training method. Its memory is also simpler than agent memory systems described in the literature: it stores JSONL records and prompt summaries rather than learned embeddings or a vector index.

4. System Overview

The reviewed repository contains three main components:

- `reflective_enginev2.py`: standalone command-line reflective loop;
- `backend/reflective_engine.py` and `backend/server.py`: Flask backend and compact-memory reflective engine;
- `frontend/`: React/Vite frontend for interacting with the backend.
- `evals/run_eval.py`: minimal baseline-versus-reflection runner that writes raw JSONL results.

The default model identifier is `gemma3:4b`, and the default endpoint is `http://localhost:11434/api/generate`. These defaults target local inference through Ollama. The model and endpoint can be changed through environment variables or backend configuration.

5. Reflective Loop Design

The standalone engine follows this high-level process:

1. Load recent memory records from JSONL.
2. Ask the model to estimate how many reflection loops are needed, bounded by the user-specified maximum.
3. Generate an initial answer.
4. Score the current answer with a separate model prompt on a 0-10 scale.
5. Store the intermediate answer and score in memory.
6. Stop if a convergence token appears, recent answers repeat, or score improvement is small after a high score.
7. Otherwise, ask the model to review the answer and suggest improvements.
8. Ask the model to produce a revised answer.
9. Run a final formatting pass when needed.

The backend implementation is similar but stores compact memory records containing answer snippets, reasoning snippets, and scores. It also includes a profile path and memory summarization function.

6. Memory Model

The current memory mechanism is deliberately lightweight. Records are stored as JSONL. The standalone engine can store full answers, while the backend engine stores truncated snippets. The backend loads recent non-empty answer snippets and renders them into a short memory summary that is inserted into future prompts.

This is persistent application memory, not model learning. The base model weights are not updated. The memory can influence later outputs only because selected records are placed back into the prompt context. This distinction matters: persistent memory can help with continuity and user preference recall, but it cannot by itself create reliable factual learning or robust generalization.

The reviewed implementation does not include TF-IDF retrieval, dense embeddings, FAISS, or another vector index. It uses direct record loading, summarization, and string similarity for repeated-answer detection.

7. Convergence And Stopping

The engine uses practical stopping heuristics rather than formal convergence guarantees. The main mechanisms are:

- explicit convergence marker: `###DONE###`;
- hard loop cap: `MAX_HARD_LOOPS`;
- user-specified maximum loops;
- repeated-answer detection using `diffLib.SequenceMatcher`;
- minimum score improvement threshold after a high self-score.

These mechanisms are useful engineering safeguards, but they should not be interpreted as mathematical proof that the model's reasoning converges to a correct answer. Natural-language outputs are not continuous numerical states, and the scoring function is itself another model call that may be wrong.

8. What Can Be Claimed From The Current Artifact

The repository supports the following claims:

- A working prototype exists for a reflective loop around an Ollama-compatible model.
- The prototype includes streaming generation, loop-count estimation, answer scoring, answer review, revision prompts, stopping heuristics, and JSONL memory.
- The project is local-first and can be configured for local model endpoints.
- The architecture is suitable for experimentation with inference-time self-evaluation.
- The repository is not yet sufficient to support quantitative benchmark claims.

The repository does not support the following claims:

- that reflection improves accuracy by a specific percentage;
- that Gemma 3 4B with reflection matches any larger model;
- that the system has a specific latency, energy, cache-hit, or memory-accuracy profile;
- that the self-scoring prompt reliably measures correctness;
- that the memory system implements semantic vector retrieval;
- that theoretical convergence has been proven for model outputs.

9. Threats To Validity

Self-evaluation can fail when the same model that produced an error is asked to detect it. A model may rate a fluent but incorrect answer highly. It may also change a correct answer into an incorrect one during revision. Reflection can amplify prompt artifacts, converge on repeated wording, or produce additional unsupported reasoning.

Factual tasks are especially limited. If the model lacks the relevant knowledge, reflection alone cannot reliably create it. Retrieval-augmented generation, tool use, or external verification would be needed for stronger factual reliability.

The current memory system can also preserve bad outputs. Without a trusted grading mechanism or user feedback controls, low-quality memory records may re-enter future prompts. The existing memory log in the reviewed repository includes empty answers and connection errors, which demonstrates the need for pruning and quality filters.

10. Reproducibility Status

The current repository is reproducible as source code and now includes a small evaluation scaffold, but it is not yet a completed empirical study. It still lacks:

- broad benchmark datasets;
- grader implementation;
- decoding configuration;
- raw output logs;
- hardware measurement logs;
- statistical analysis scripts.

The scaffold in `evals/run_eval.py` can compare direct single-pass answers against reflective answers on JSONL tasks and save raw outputs. The included `evals/sample_tasks.jsonl` is only a smoke-test set, not a benchmark. This means the artifact can begin collecting evidence, but quantitative research claims still cannot be made until larger task sets and logged runs exist.

11. Evaluation Plan

A defensible evaluation should include:

- exact model tag and quantization, for example Ollama `gemma3:4b Q4_K_M`;
- fixed sampling parameters;
- public benchmark subsets with explicit licenses;
- prompt templates for baseline and reflective conditions;
- exact-answer grading for arithmetic and multiple-choice tasks;
- human or rubric-based grading only where deterministic grading is impossible;
- raw generations saved for every example;
- per-category accuracy, confidence intervals, and error analysis;
- latency and token throughput measured from logs;
- energy measurement only with a documented power measurement method.

The minimum useful comparison is not against a larger model. It is the same local model under two conditions:

- baseline: direct single-pass answer;
- reflective: same model and decoding settings with reflection enabled.

Only after that comparison is reproducible should the project compare against larger models or claim efficiency benefits.

12. Improvement Metrics

Once raw evaluation results exist, improvements can be computed with the following equations. Let N be the number of graded tasks, B be the number answered correctly by the baseline, and R be the number answered correctly by the reflective system:

- Baseline accuracy: $A_{\text{base}} = B / N$
- Reflective accuracy: $A_{\text{reflect}} = R / N$

- Absolute improvement in percentage points: $AII = (A_{reflect} - A_{base}) * 100$
- Relative performance gain: $RPG = ((A_{reflect} - A_{base}) / A_{base}) * 100$
- Baseline error rate: $E_{base} = 1 - A_{base}$
- Reflective error rate: $E_{reflect} = 1 - A_{reflect}$
- Error reduction rate: $ERR = ((E_{base} - E_{reflect}) / E_{base}) * 100$

For runtime, let L_{base} be average baseline latency and $L_{reflect}$ be average reflective latency:

- Latency overhead: $Overhead = ((L_{reflect} - L_{base}) / L_{base}) * 100$

The repository includes `evals/analyze_results.py` to compute these metrics from JSONL output produced by `evals/run_eval.py`.

<!-- METRICS_START -->

Exact improvement scores are not reported yet because no completed graded evaluation run is included in the repository snapshot. After running `evals/run_eval.py`, use `evals/update_paper_metrics.py` to insert exact scores here.

<!-- METRICS_END -->

13. Discussion

Reflective Engine is valuable as a transparent software pattern. It exposes the steps that many users already perform manually: ask, inspect, challenge, revise, and finalize. Placing those steps into an application loop makes the process configurable and observable.

The key engineering tradeoff is cost versus quality. Reflection adds calls to the model, so it must consume more time and tokens than a single-pass answer. Whether that cost improves output quality is task-dependent and must be measured. The best use cases are likely tasks where the base model can recognize and correct its own mistakes when prompted, such as formatting, consistency checks, simple code review, and some multi-step explanations. The weakest use cases are tasks requiring unknown facts, precise citations, or external state.

14. Conclusion

Reflective Engine demonstrates a local application architecture for iterative self-evaluation around an Ollama-compatible language model. The implemented system provides reflection loops, self-scoring, revision prompts, convergence markers, repeated-answer detection, and persistent JSONL memory. These are real, inspectable software contributions.

The current artifact should not be presented as evidence of specific benchmark gains. Its honest research status is a prototype and evaluation scaffold. With a benchmark harness, raw logs, deterministic grading, and statistical analysis, it could become the basis for a stronger empirical paper on when inference-time reflection helps and when it fails.

References

- Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS, 2022. <https://arxiv.org/abs/2201.11903>
- Xuezhi Wang et al. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." 2022. <https://arxiv.org/abs/2203.11171>
- Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." 2023. <https://arxiv.org/abs/2303.11366>
- Shunyu Yao et al. "Tree of Thoughts: Deliberate Problem Solving with Large Language Models." 2023. <https://arxiv.org/abs/2305.10601>

- Aman Madaan et al. "Self-Refine: Iterative Refinement with Self-Feedback." 2023. <https://arxiv.org/abs/2303.17651>
- Yuntao Bai et al. "Constitutional AI: Harmlessness from AI Feedback." 2022. <https://arxiv.org/abs/2212.08073>
- Google Developers Blog. "Introducing Gemma 3: The Developer Guide." 2025.
<https://developers.googleblog.com/en/introducing-gemma3/>
- Ollama model registry. gemma3 : 4b. <https://registry.ollama.ai/library/gemma3%3A4b>
- Hugging Face model card. meta-llama/Llama-2-13b. <https://huggingface.co/meta-llama/Llama-2-13b>
- Hugo Touvron et al. "Llama 2: Open Foundation and Fine-Tuned Chat Models." 2023.
<https://arxiv.org/abs/2307.09288>